

Llamadas con streaming y sin streaming

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 7 de 12 · Notas en español (es-ES)

Hasta mediada la sesión, cada demo usaba `messages.create` y esperaba un **Message** terminado. Lalit preguntó cómo Claude Chat muestra tokens según aparecen. Respuesta de Vijay: usar el helper `stream` del SDK o `stream=true` en `create`. Mismo modelo, distinta entrega: un JSON completo frente a una secuencia de **eventos enviados por el servidor**.

Sin streaming: un Message cuando acaba la generación

Con `create` por defecto, el cuerpo HTTP es el mensaje de asistente entero. Si la generación tardara «un minuto» (ejemplo exagerado de Vijay), la UI se quedaría en blanco ese minuto. Luego parseáis `content` como bloques de texto completos (`type: text`). Es más simple para lotes, tests y cualquier cliente que no pueda pintar texto parcial. Es una espera **síncrona**: la llamada vuelve cuando el modelo termina (o choca con `max_tokens / stop`).

Para qué sirve el streaming

- Latencia percibida: mostrar tokens en cuanto existen para no perder al usuario.
- Respuestas largas: no bufferizar todo el ensayo en el SDK antes de pintar la primera frase.
- Productos tipo chat: Claude Chat y los agentes de IDE se sienten rápidos porque hacen streaming; vuestra app de API se sentirá «colgada» si solo usáis `create` bloqueante.

El streaming no hace el modelo más listo ni más barato por sí solo. Seguis pagando tokens de salida. Sí asumís complejidad de eventos.

Dos entradas en el SDK

Mecanismo (lenguaje de clase)	Idea	Uso típico
<code>messages.create</code> sin <code>stream</code>	Objeto Message completo	Scripts, evals, backends simples
<code>stream=true</code> en <code>create</code>	El mismo método, HTTP en flujo	Cuando queréis <code>create</code> pero bytes incrementales
<code>messages.stream</code> / helper	API de streaming dedicada en el SDK	Callbacks, context managers

Vijay dijo que cada vía tiene ventajas: ¿queréis un objeto cerrado o un iterador de eventos? Confirmad los nombres exactos en el SDK Python/TS vigente — se han renombrado más de una vez.

Eventos que se mostraron

En el cable, el streaming es una serie de eventos. El notebook los imprimía todos y luego filtraba el texto. Recordad los nombres de clase y mapeadlos a la documentación:

1. **message_start** — ya hay un id de mensaje; el contenido aún está vacío. «Voy a enviaros datos».
2. **content_block_start** — empieza un bloque nuevo (casi siempre texto).
3. **content_block_delta** — carga incremental. Los deltas se acumulan. El texto llega como piezas **text_delta**, aún no como bloque `type: text` cerrado.
4. **content_block_stop** — ese bloque está completo.
5. **message_stop** (en clase: parada del mensaje en bruto) — el flujo de este mensaje ha terminado.

El usuario final no debe ver nombres de eventos. Vuestra UI concatena valores `text_delta`. Si ignoráis todo lo que no sea texto, perderéis errores, usage y (luego) bloques de tools o thinking.

EL «PENSANDO...» DE VS CODE NO ES ESTE STREAM

Mritunjay preguntó en qué bloque está el cromado humanizado de «thinking / working» en VS Code. Vijay: esa UX es **Agent SDK** / capa de producto, no el stream crudo de Messages. El Agent SDK sigue apoyándose en estos deltas para los tokens reales.

Los stop reason siguen aplicando

Con o sin streaming, la generación puede acabar por fin de turno, una **secuencia de parada** (capítulo siguiente) o **max_tokens**. El streaming solo entrega antes los mismos tokens. Si salta `max_tokens`, los últimos deltas pueden ser una frase cortada — el JSON incompleto lo gestionáis vosotros.

Batches (solo de oídas)

El recurso messages también expone **batches** para trabajos fuera de línea. El foco de esta hora era create interactivo frente a stream, no diseñar un pipeline por lotes. Sabed que batch existe para no confundirlo con streaming.

DOCUMENTACIÓN POR ENCIMA DEL VOLCADO DE EVENTOS

Las cadenas de tipo de evento de agosto de 2026 pueden haber cambiado. Usad la [documentación de streaming](#) como clave de corrección.

Lista de implementación

- Create bloqueante: `parsead content[ ].text`.
- Streaming: suscribíos a deltas; unid texto; leed usage/stop finales en el evento terminal o en un Message de cierre.
- No asumáis que el streaming reduce tokens facturados.

- El tiempo hasta el primer token es la métrica de UX que mejora el streaming.

Documentación oficial

- **Streaming**

<https://docs.claude.com/en/docs/build-with-claude/streaming>

Tipos de evento, SDKs y comportamiento SSE.

- **Messages API**

<https://docs.claude.com/en/api/messages>

El campo `stream` en `create`.

- **Visión general de la API**

<https://docs.claude.com/en/api/overview>

Cómo los clientes envuelven HTTP, también en streaming.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.